Assignment 4: Data Wrangling (Fall 2025)

Kaitlyn Kukula

September 25, 2025

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Wrangling

Directions

1. Rename this file `<FirstLast>_A04_DataWrangling.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change "Student Name" on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. Ensure that code in code chunks does not extend off the page in the PDF.

Set up your session

1a. Load the `tidyverse`, `lubridate`, and `here` packages into your session.

1b. Check your working directory.

1c. Read in all four raw data files associated with the EPA Air dataset, being sure to set string columns to be read in as factors. See the README file for the EPA air datasets for more information (especially if you have not worked with air quality data previously).

2. Add the appropriate code to reveal the dimensions of the four datasets.

```
#1a: load relevant packages into session

library(tidyverse)
library(lubridate)
library(here)

#1b: check working directory
here()
```

```
## [1] "/Users/kaitlynkukula/EDE_Fall2025"
```

```
getwd()
```

```
## [1] "/Users/kaitlynkukula/EDE_Fall2025"
```

```
#1c: read in air quality datasets

EPAair_O3_NC2018 <- read.csv(
  file = here("Data/Raw/EPAair_O3_NC2018_raw.csv"),
  stringsAsFactors = TRUE)

EPAair_O3_NC2019 <- read.csv(
  file = here("Data/Raw/EPAair_O3_NC2019_raw.csv"),
  stringsAsFactors = TRUE)

EPAair_PM25_NC2018 <- read.csv(
  file = here("Data/Raw/EPAair_PM25_NC2018_raw.csv"),
  stringsAsFactors = TRUE)

EPAair_PM25_NC2019 <- read.csv(
  file = here("Data/Raw/EPAair_PM25_NC2019_raw.csv"),
  stringsAsFactors = TRUE)

#2: get dimensions of the datasets

dim(EPAair_O3_NC2018)
```

```
## [1] 9737   20
```

```
dim(EPAair_O3_NC2019)
```

```
## [1] 10592    20
```

```
dim(EPAair_PM25_NC2018)
```

```
## [1] 8983   20
```

```
dim(EPAair_PM25_NC2019)
```

```
## [1] 8581   20
```

> All four datasets should have the same number of columns but unique record counts (rows). Do your datasets follow this pattern?

Answer: Yes, each dataset has 20 variables with varying numbers of obervations.

Wrangle individual datasets to create processed files.

3. Change the Date columns to be date objects.
4. Select the following columns: Date, DAILY_AQI_VALUE, Site.Name, AQS_PARAMETER_DESC, COUNTY, SITE_LATITUDE, SITE_LONGITUDE

5. For the PM2.5 datasets, fill all cells in AQS_PARAMETER_DESC with "PM2.5" (all cells in this column should be identical).

6. Save all four processed datasets in the Processed folder. Use the same file names as the raw files but replace "raw" with "processed".

```
#3: Convert the "Date" variables in each of the four datasets from factors to date types

EPAair_O3_NC2018$Date <- mdy(EPAair_O3_NC2018$Date)
EPAair_O3_NC2019$Date <- mdy(EPAair_O3_NC2019$Date)
EPAair_PM25_NC2018$Date <- mdy(EPAair_PM25_NC2018$Date)
EPAair_PM25_NC2019$Date <- mdy(EPAair_PM25_NC2019$Date)

#4: Select the columns of interest from each raw dataframe create a "processed" dataframe

EPAair_O3_NC2018_processed <-
  EPAair_O3_NC2018 %>%
  select(c("Date", "DAILY_AQI_VALUE", "Site.Name", "AQS_PARAMETER_DESC", "COUNTY",
           "SITE_LATITUDE", "SITE_LONGITUDE"))

EPAair_O3_NC2019_processed <-
  EPAair_O3_NC2019 %>%
  select(c("Date", "DAILY_AQI_VALUE", "Site.Name", "AQS_PARAMETER_DESC", "COUNTY",
           "SITE_LATITUDE", "SITE_LONGITUDE"))

EPAair_PM25_NC2018_processed <-
  EPAair_PM25_NC2018 %>%
  select(c("Date", "DAILY_AQI_VALUE", "Site.Name", "AQS_PARAMETER_DESC", "COUNTY",
           "SITE_LATITUDE", "SITE_LONGITUDE"))

EPAair_PM25_NC2019_processed <-
  EPAair_PM25_NC2019 %>%
  select(c("Date", "DAILY_AQI_VALUE", "Site.Name", "AQS_PARAMETER_DESC", "COUNTY",
           "SITE_LATITUDE", "SITE_LONGITUDE"))


#5: Populate each AQS_PARAMETER_DESC column in the two PM25 datasets with "PM2.5"

EPAair_PM25_NC2018_processed$AQS_PARAMETER_DESC = "PM2.5"
EPAair_PM25_NC2019_processed$AQS_PARAMETER_DESC = "PM2.5"

#6: Create a new .csv file in the Processed folder for each of the datasets

write.csv(EPAair_O3_NC2018_processed,
          row.names = FALSE,
          file = here("Data/Processed/EPAair_O3_NC2018_processed"))

write.csv(EPAair_O3_NC2019_processed,
          row.names = FALSE,
          file = here("Data/Processed/EPAair_O3_NC2019_processed"))

write.csv(EPAair_PM25_NC2018_processed,
          row.names = FALSE,
          file = here("Data/Processed/EPAair_PM25_NC2018_processed"))
```

```
write.csv(EPAair_PM25_NC2019_processed,
          row.names = FALSE,
          file = here("Data/Processed/EPAair_PM25_NC2019_processed"))
```

Combine datasets

7. Combine the four datasets with `rbind`. Make sure your column names are identical prior to running this code.
8. Wrangle your new dataset with a pipe function (%>%) so that it fills the following conditions:

- Include only sites that the four data frames have in common:

"Linville Falls", "Durham Armory", "Leggett", "Hattie Avenue",
"Clemmons Middle", "Mendenhall School", "Frying Pan Mountain", "West Johnston Co.", "Garinger High School", "Castle Hayne", "Pitt Agri. Center", "Bryson City", "Millbrook School"

(the function `intersect` can figure out common factor levels - but it will include sites with missing site information, which you don't want...)

- Some sites have multiple measurements per day. Use the split-apply-combine strategy to generate daily means: group by date, site name, AQS parameter, and county. Take the mean of the AQI value, latitude, and longitude.
- Add columns for "Month" and "Year" by parsing your "Date" column (hint: `lubridate` package)
- Hint: the dimensions of this dataset should be 14,752 x 9.

9. Spread your datasets such that AQI values for ozone and PM2.5 are in separate columns. Each location on a specific date should now occupy only one row.
10. Call up the dimensions of your new tidy dataset.
11. Save your processed dataset with the following file name: "EPAair_O3_PM25_NC1819_Processed.csv"

```
#7: Combine the four datasets using "rbind"

EPAair_data <- rbind(EPAair_O3_NC2018_processed,
                     EPAair_O3_NC2019_processed,
                     EPAair_PM25_NC2018_processed,
                     EPAair_PM25_NC2019_processed)

#8: Wrangle the data to the specifications above

# Use the filter() function to select only the rows that contain a site of interest
EPAair_wrangled <-
  EPAair_data %>%
  filter(Site.Name %in% c("Linville Falls", "Durham Armory", "Leggett", "Hattie Avenue",
  "Clemmons Middle", "Mendenhall School", "Frying Pan Mountain",
  "West Johnston Co.", "Garinger High School", "Castle Hayne",
  "Pitt Agri. Center", "Bryson City", "Millbrook School"))

# Apply the split-apply-combine strategy to generate daily means
```

```r
EPAair_summaries <-
  EPAair_wrangled %>%
  # Group by: date, site name, AQS parameter, and county
  group_by(Site.Name, Date, AQS_PARAMETER_DESC, COUNTY) %>%
  # Calculate mean: AQI value, latitude, and longitude
  summarise(meanAQI = mean(DAILY_AQI_VALUE),
            meanlat = mean(SITE_LATITUDE),
            meanlong = mean(SITE_LONGITUDE))
```

```
## `summarise()` has grouped output by 'Site.Name', 'Date', 'AQS_PARAMETER_DESC'.
## You can override using the `.groups` argument.
```

```r
# Add columns for "Month" and "Year" by parsing your "Date" column
EPAair_summaries <-
  EPAair_summaries %>%
  mutate(Month = month(Date)) %>%  # Create a column for the month
  mutate(Year = year(Date)) %>%  # Create a column for the year
  select(Site.Name:Date, Month, Year, AQS_PARAMETER_DESC:meanlong) # Rearrange order of columns

# Get dimensions of the summaries dataframe with the added month and year
dim(EPAair_summaries)
```

```
## [1] 14752     9
```

```r
#9: Spread the dataset using pivot_wider() such that AQI values for ozone and
# PM2.5 are in separate columns

# Create the pivoted dataset
EPAair_O3_PM25_NC1819_Processed <-
  EPAair_summaries %>%
  pivot_wider(names_from = "AQS_PARAMETER_DESC", # what you want the categories to be
              values_from = "meanAQI") # the values you want to sort into categories

#10: Get dimensions of the new dataset (Reveals that the values from AQQS_PARAMETER_DESC and
# meanAQI were converted to two new columns

dim(EPAair_O3_PM25_NC1819_Processed)
```

```
## [1] 8976    9
```

```r
colnames(EPAair_O3_PM25_NC1819_Processed)
```

```
## [1] "Site.Name" "Date"      "Month"     "Year"      "COUNTY"    "meanlat"
## [7] "meanlong"  "PM2.5"     "Ozone"
```

```r
#11: Save the dataset by creating a .csv in the Processed Data folder

write.csv(EPAair_O3_PM25_NC1819_Processed,
          file = here("Data/Processed/EPAair_O3_PM25_NC1819_Processed.csv") )
```

Generate summary tables

12. Use the split-apply-combine strategy to generate a summary data frame. Data should be grouped by site, month, and year. Generate the mean AQI values for ozone and PM2.5 for each group. Then, add a pipe to remove instances where mean **ozone** values are not available (use the function `drop_na` in your pipe). It's ok to have missing mean PM2.5 values in this result.

13. Call up the dimensions of the summary dataset.

```
#12: Generate a summary data frame

EPAair_O3_PM25_NC1819_Summary <-
  EPAair_O3_PM25_NC1819_Processed %>%
  group_by(Site.Name, Month, Year) %>% # Create the groups
  summarise(meanAQI_ozone = mean(Ozone),
            meanAQI_PM25 = mean(PM2.5)) %>% # Calculate the statistics
  drop_na(meanAQI_ozone) # Remove NA entries in the meanAQI_ozone column
```

```
## `summarise()` has grouped output by 'Site.Name', 'Month'. You can override
## using the `.groups` argument.
```

```
#13: Get the dimensions of the new dataset

dim(EPAair_O3_PM25_NC1819_Summary)
```

```
## [1] 182   5
```

14. Why did we use the function `drop_na` rather than `na.omit`? Hint: replace `drop_na` with `na.omit` in part 12 and observe what happens with the dimensions of the summary date frame.

    Answer: na.omit() will remove any row, from any column, that has an NA value, so you should use drop.na when you want to choose which column you wish to remove NA values from.